NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

Experiment 2:

Aim: To containerise the application using docker

This experiment maps to LO4.

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory:

Docker is an open-source containerization platform that enables applications to run in isolated environments called **containers**. A container includes the application along with all its required libraries, dependencies, and configuration files, ensuring consistent execution across different systems. Since containers share the host operating system's kernel, they are lightweight, portable, and start much faster than traditional virtual machines.

A Docker image is a read-only template that contains everything needed to create a container, including the operating system, software packages, and application files. Images are stored in repositories such as Docker Hub and can be downloaded using the docker pull command. Once an image is available locally, it can be used to create one or more containers.

Docker provides several commands for managing images and containers. The docker --version command is used to verify the installed Docker version, while docker images display all locally available Docker images. The docker search command allows users to search Docker Hub for official or community-maintained images, and docker pull downloads the required image from the repository.

Containers are created and started using the docker run command. Running a container interactively (docker run -it ubuntu) opens a Linux shell inside the container, allowing users to execute Linux commands as if they were working on a standalone Ubuntu system. Commands such as pwd display the current working directory, ls lists the files and directories, mkdir creates a new directory, cd changes the current directory, touch creates an empty file, echo writes text into a file, and cat displays the contents of the file.

These basic Docker and Linux commands help users understand how containers work, how to interact with a running container, and how to perform file and directory operations within an isolated environment. This forms the foundation for deploying, testing, and managing applications using Docker in real-world DevOps workflows.

NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

Docker Installation Steps

1. Check system requirements: Ensure your operating system supports Docker and virtualization.
2. Enable virtualization (Windows): Turn on Intel VT-x or AMD-V in the BIOS/UEFI settings.
3. Install WSL 2 (Windows only): WSL 2 provides the Linux environment required by Docker Desktop.
4. Download and install Docker Desktop: Install Docker Desktop from the official Docker website and follow the setup instructions.
5. Restart your computer: Restart if prompted to complete the installation.
6. Launch Docker Desktop: Open Docker Desktop and wait for the Docker Engine to start.

What is a docker image?

A Docker image is a read-only template used to create Docker containers. It contains everything required to run an application, including the operating system, application code, libraries, dependencies, configuration files, and runtime environment. Docker images are stored in repositories such as Docker Hub and can be downloaded using the docker pull command. Since images are portable and platform-independent, they ensure that applications run consistently across different environments.

A Docker image itself cannot execute applications. When the docker run command is used, Docker creates a container, which is a running instance of the image. Multiple containers can be created from the same image, making Docker an efficient solution for application deployment, testing, and scaling. Images are also built in layers, which reduces storage requirements and speeds up image downloads and updates by reusing common layers.

What is containerization?

Containerization is a virtualization technology that packages an application along with all its dependencies, libraries, configuration files, and runtime environment into a single, isolated unit called a container. This ensures that the application runs consistently across different environments, regardless of the underlying operating system or hardware.

Unlike traditional virtual machines, containers share the host operating system's kernel, making them lightweight, fast, and resource-efficient. Containers start within seconds, require less

NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

memory, and allow multiple applications to run independently on the same system without conflicts. Docker is one of the most widely used platforms for containerization, enabling developers to build, deploy, and manage applications efficiently. Containerization improves application portability, scalability, and reliability, making it an essential technology in modern software development and DevOps practices.

What is the difference between docker image and docker container?

Docker Image	Docker Container
A read-only template used to create containers.	A running instance of a Docker image.
Contains application code, libraries, dependencies, and configuration files.	Contains the running application along with its processes and data.
Cannot execute applications by itself.	Executes the application inside an isolated environment.
Created using a Dockerfile.	Created using the docker run command.
Stored in image repositories such as Docker Hub.	Exists only while it is created on the host system.
Multiple containers can be created from a single image.	Each container is an independent instance of the image.
Read-only and cannot be modified directly.	Read-write; changes can be made while the container is running.
Example: ubuntu, nginx, python, mysql images.	Example: A running Ubuntu container created using docker run -it ubuntu.

Implementation:

1)check docker version

```
C:\Users\15L>docker --version
Docker version 25.0.3, build 4debf41
```

NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

2)images

```
PS C:\Users\Tejas> docker images
REPOSITORY            TAG         IMAGE ID       CREATED       SIZE
4score-hackathon-web  latest      bce374721d3e   13 days ago   559MB
4score-hackathon-api  latest      ab5cf9310e9b   13 days ago   518MB
ubuntu                latest      3131b4cc82a7   2 weeks ago   157MB
postgres              16-alpine   57c72fd2a128   3 weeks ago   419MB
PS C:\Users\Tejas> docker search python
NAME                      DESCRIPTION                                     STARS     OFFICIAL
python                    Python is an interpreted, interactive, objec…   10498     [OK]
circleci/python           Python is an interpreted, interactive, objec…   116
cimg/python                                                               25
demisto/python            The Demisto base python image                   5
ubuntu/python             A chiselled Ubuntu rock with the Python runt…   33
okteto/python                                                             0
paketobuildpacks/python                                                   0
activestate/python        ActiveState's customizable, low-to-no vulner…   11
cleanstart/python         Deploy, scale, and secure critical infrastru…   1
intel/python                                                              5
```

3)

```
PS C:\Users\Tejas> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
d26f27cc8c41: Pull complete
062e450697fa: Pull complete
2bedaf25031a: Pull complete
cacfcdd01f30: Pull complete
82454cdbf456: Pull complete
3c7ab7949321: Pull complete
b6698f04e005: Pull complete
Digest: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
PS C:\Users\Tejas> docker images
REPOSITORY            TAG         IMAGE ID       CREATED       SIZE
4score-hackathon-web  latest      bce374721d3e   13 days ago   559MB
4score-hackathon-api  latest      ab5cf9310e9b   13 days ago   518MB
nginx                 latest      5a88c9c45479   13 days ago   238MB
ubuntu                latest      3131b4cc82a7   2 weeks ago   157MB
postgres              16-alpine   57c72fd2a128   3 weeks ago   419MB
PS C:\Users\Tejas> docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
```

4)

NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

```
PS C:\Users\Tejas> docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
PS C:\Users\Tejas> docker run -it --name ubuntu1 ubuntu bash
root@a6e1cf919000:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
root@a6e1cf919000:/# mkdir docker_lab
root@a6e1cf919000:/# cd docker_lab
root@a6e1cf919000:/docker_lab# touch hello.txt
root@a6e1cf919000:/docker_lab# echo "Docker Practical" > hello.txt
root@a6e1cf919000:/docker_lab# cat hello.txt
Docker Practical
root@a6e1cf919000:/docker_lab# exit
exit
PS C:\Users\Tejas> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED          STATUS          PORTS     NAMES
e818f67135d5   ubuntu    "bash"    14 minutes ago   Up 14 minutes             ubuntu-terminal
```

```
PS C:\Users\Tejas> docker start ubuntu1
ubuntu1
PS C:\Users\Tejas> docker exec -it ubuntu1 bash
root@a6e1cf919000:/# docker stop ubuntu1
bash: docker: command not found
root@a6e1cf919000:/#
PS C:\Users\Tejas> docker inspect ubuntu1
[
    {
        "Id": "a6e1cf9190003ac496a5e06ccee66efe95ec42b9317c155c925369a29e903ab6",
        "Created": "2026-07-29T08:12:16.590410438Z",
        "Path": "bash",
        "Args": [],
        "State": {
            "Status": "exited",
            "Running": false,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 0,
            "ExitCode": 137,
            "Error": "",
            "StartedAt": "2026-07-29T08:15:31.06606832Z",
            "FinishedAt": "2026-07-29T08:16:19.58371707Z"
        },
        "Image": "sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb",
```

NAME: Tejas Mandhyani
T23 2511135
SUBJECT: ASD&D
DATE: 29/07/2026

```
PS C:\Users\Tejas> docker start ubuntu1
ubuntu1
PS C:\Users\Tejas> docker exec -it ubuntu1 bash
root@a6e1cf919000:/# echo "Hello"
Hello
root@a6e1cf919000:/# exit
exit
PS C:\Users\Tejas> docker logs ubuntu1
root@a6e1cf919000:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
root@a6e1cf919000:/# mkdir docker_lab
root@a6e1cf919000:/# cd docker_lab
root@a6e1cf919000:/docker_lab# touch hello.txt
root@a6e1cf919000:/docker_lab# echo "Docker Practical" > hello.txt
root@a6e1cf919000:/docker_lab# cat hello.txt
Docker Practical
root@a6e1cf919000:/docker_lab# exit
exit
```

```
PS C:\Users\Tejas> docker ps
CONTAINER ID   IMAGE     COMMAND             CREATED         STATUS             PORTS     NAMES
7803d28c64a2   ubuntu    "tail -f /dev/null"   16 seconds ago   Up 15 seconds              ubuntu-bg
a6e1cf919000   ubuntu    "bash"              9 minutes ago   Up About a minute          ubuntu1
e818f67135d5   ubuntu    "bash"              20 minutes ago  Up 20 minutes              ubuntu-terminal
```

Conclusion:

In this experiment, we learned the fundamentals of Docker by performing basic image and container operations. We verified the Docker installation, searched for and downloaded Docker images from Docker Hub, listed locally available images, and created an Ubuntu container using a Docker image. Inside the container, we executed basic Linux commands to navigate directories, create folders and files, write data to a file, and verify its contents. This experiment helped us understand the concepts of Docker images, containers, and containerization, as well as how Docker provides an isolated and consistent environment for running applications efficiently.